



Desarrollo de Aplicaciones Web

Proyecto

***StockTrace* - Aplicación Web para Gestión de Inventario**

Autor

Rodrigo Sánchez Martínez

Tutor

Mario Álvarez Fernández

06/05/2026

1 Contenido

2	Introducción	2
2.1	Presentación del proyecto	2
3	Metodología y temporalización	3
3.1	Metodología de trabajo	3
3.2	Organización temporal del proyecto	4
4	Objetivos, requisitos y actores	5
4.1	Objetivo general y específicos del proyecto	5
4.2	Requisitos funcionales y no funcionales	6
4.3	Actores	7
5	Diseño y navegación	8
5.1	Prototipos	8
5.2	Guía de Estilo	10
5.3	Mapa de navegación	10
6	Diagrama de Casos de Uso	11
6.1	Diagrama de Casos de Uso	11
6.2	Tablas de Especificación	13
7	Diagramas de Secuencia	16
8	Diagramas de Clase	17
8.1	Diagrama de clases del modelo de datos	17
8.2	Diagrama de clases de diseño (arquitectura del backend)	18
9	Modelo Entidad-Relación	19
10	Marketing	21
10.1	Análisis de Macroentorno y Microentorno	21
10.2	DAFO	24
10.3	Plan de Marketing	31
10.4	Valoración del proyecto	32
11	Pruebas	35
12	Tecnologías utilizadas	36
12.1	Frontend (cliente)	36
12.2	Backend (servidor)	37
12.3	Base de datos	38
12.4	Herramientas de desarrollo y despliegue	38
12.5	Nota sobre la elección de JavaScript frente a Python	38
13	Manuales	39
13.1	Manual de usuario	39
13.2	Manual de programador y despliegue	41
14	Conclusión	41
14.1	Conclusiones	41
14.2	Líneas de trabajo futuras	44
15	Bibliografía	47

2 Introducción

El sistema se concibe como una solución de trazabilidad que trata de solucionar los problemas de integridad de datos en un almacén. El objetivo es la eliminación del error humano, que en sistemas manuales puede superar el 20%, reduciéndolo a menos del 1% usando la captura automatizada de datos. La arquitectura se basa en un ecosistema de código abierto, seleccionando Node.js con Express para el backend, React para la interfaz de usuario y MySQL para la base de datos.

2.1 Presentación del proyecto

StockTrace es una aplicación web orientada a automatizar la gestión del inventario en PYMEs mediante códigos QR, ofreciendo un control en tiempo real de todas las transacciones y una visibilidad completa sobre el estado del stock. Sus funcionalidades principales incluyen:

- Catálogo de productos con generación automática de QR: cada artículo recibe un identificador único y un código QR que lo vincula a su ficha detallada.
- Registro de movimientos en tiempo real: entradas, salidas y ajustes que actualizan instantáneamente las existencias.
- Sistema de alertas: avisos automáticos al administrador cuando un producto desciende por debajo del umbral mínimo configurado.
- Clasificación ABC dinámica: análisis del principio de Pareto sobre los movimientos recientes para categorizar los productos según su impacto económico.
- Control de acceso basado en roles: tres perfiles —administrador, operador y auditor— con permisos jerárquicos diferenciados.
- Exportación de reportes e historial de movimientos para auditoría y control externo.

La aplicación ha sido diseñada priorizando la usabilidad desde dispositivos móviles, de modo que un operador pueda escanear códigos con la cámara de su smartphone y registrar un movimiento en cuestión de segundos, sin necesidad de adquirir escáneres industriales ni otro hardware costoso.

En esta memoria se recoge el proceso completo de desarrollo, desde el análisis del entorno y la definición de requisitos hasta el diseño de la interfaz, la implementación

del backend y el frontend, las pruebas de calidad y los manuales de usuario y de programador. Todo ello se ha organizado siguiendo una metodología ágil Kanban, que ha permitido mantener un flujo de trabajo constante y adaptativo a lo largo de las fases del proyecto.

3 Metodología y temporalización

3.1 Metodología de trabajo

Para la gestión del desarrollo del software se ha seleccionado la metodología Kanban, una aproximación ágil que permite mantener un sistema de trabajo flexible y adaptarse rápidamente a la variación de los requerimientos, algo especialmente relevante en un proyecto orientado a PYMEs donde las necesidades pueden evolucionar a lo largo del ciclo de vida.

La utilización de Kanban se concreta en un tablero visual donde se disponen las tareas en columnas que representan su estado: Backlog, Análisis, Desarrollo, Pruebas y Hecho. Cada tarea se materializa como una tarjeta que avanza por el tablero a medida que progresa su implementación. Esta representación permite monitorizar de un vistazo la productividad del equipo, identificar posibles cuellos de botella y mantener un flujo de trabajo constante.

Un principio fundamental de Kanban es la limitación del trabajo en curso (WIP, por sus siglas en inglés). Al restringir el número de tareas que pueden estar simultáneamente en cada columna, se evita la sobrecarga de los recursos y se garantiza que cada funcionalidad se complete antes de abordar la siguiente. Esta práctica ha resultado especialmente valiosa en este proyecto, dado que el 86 % de los costes se concentran en el tiempo de desarrollo; cualquier desviación en la planificación habría repercutido directamente en el presupuesto final.

La elección de Kanban frente a otras metodologías ágiles como Scrum responde a la naturaleza del proyecto. Al tratarse de un desarrollo individual, las iteraciones con sprints de duración fija resultaban menos apropiadas que un flujo continuo donde las tareas pudieran incorporarse o reordenarse según las prioridades detectadas durante la implementación.

3.2 Organización temporal del proyecto

El ciclo de vida completo del software se ha planificado con un margen de 17 semanas, desde el análisis inicial de requerimientos hasta la finalización de los manuales y la memoria. La distribución temporal se ha organizado en las fases que se detallan a continuación, cada una de las cuales agrupa tareas afines y ha estado sujeta al control visual del tablero Kanban.

Fase	Semanas	Tareas principales
Análisis y viabilidad	1-2	Análisis del macroentorno y microentorno (PESTEL), matriz DAFO, definición de objetivos SMART, requisitos funcionales y no funcionales, identificación de actores.
Diseño	3-4	Prototipado de alta fidelidad en Figma, definición de la guía de estilo «Soft Industrial», mapa de navegación, diagramas de casos de uso, modelo entidad-relación y esquema de base de datos.
Implementación del backend	5-9	Configuración del entorno (Node.js, Express, MySQL), desarrollo modular de la API REST (autenticación, CRUD de productos, movimientos, alertas, clasificación ABC, dashboard), control de acceso basado en roles, validaciones y seguridad.
Implementación del frontend	9-14	Inicialización del proyecto React, componentes reutilizables (tarjetas, tablas, escáner QR), páginas principales (dashboard, catálogo, movimientos, alertas, clasificación ABC, reportes, administración de usuarios, perfil), integración con la API, diseño responsive y aplicación de la guía de estilo.
Pruebas	15-16	Tests unitarios, de integración, funcionales, de usabilidad, de seguridad y de rendimiento. Validación de flujos completos con los tres roles de usuario.
Documentación y cierre	17	Redacción del manual de usuario, manual de programador y despliegue, conclusión, líneas de trabajo futuras y preparación de la memoria final.

La estimación de 17 semanas se corresponde con la valoración económica del proyecto presentada en el apartado 10.4, donde el coste del desarrollo se calcula sobre esa misma base temporal. El seguimiento semanal mediante el tablero Kanban ha permitido verificar que el ritmo de avance se mantenía dentro de lo planificado y que las posibles desviaciones se corregían antes de que impactaran en la fecha de finalización prevista.

4 Objetivos, requisitos y actores

4.1 Objetivo general y específicos del proyecto

4.1.2. Objetivo general

El objetivo general es implementar un sistema web bien diseñado para manejar el stock de manera que permita la administración del inventario con un control preciso de todos los movimientos por medio del uso de códigos QR generados por la propia aplicación, otorgando una visión en tiempo real de todas las operaciones realizadas para así optimizar la eficiencia y la rentabilidad de las PYMEs.

4.1.3. Objetivos específicos

Los siguientes objetivos son más específicos y desglosan el objetivo principal en funciones concretas:

- Conseguir una reducción casi total de la tasa de errores desde los sistemas tradicionales de manejo de inventario, idealmente hasta un 1% como máximo por medio de la automatización del registro de todas las transacciones usando códigos QR.
- Aumentar la eficiencia y aumentar la velocidad de todo el proceso logístico que antes se realizaba manualmente. Esto también permitiría relocalizar la mano de obra que se encargaba de estos procesos a otras áreas para incrementar aún más la eficiencia.
- Incluir en cada código QR todos los datos necesarios de cada elemento del inventario para asegurar un sistema de seguimiento que permita rastrear cualquier producto. Implementar un sistema automatizado para notificar a los administradores cuándo es necesario hacer restock de un producto.

- Eliminar la necesidad de invertir en productos de hardware costosos para escanear los códigos QR. Esto se conseguiría haciendo accesible la identificación de los códigos QR permitiendo el uso de la cámara de dispositivos móviles comunes para identificarlos, prescindiendo así de productos más específicos y caros que llevasen a cabo esta función.

4.2 Requisitos funcionales y no funcionales

4.2.1. Requisitos funcionales

Los requisitos funcionales describen los servicios, procesos y funciones que son esenciales para solventar el problema de la administración del stock. Los siguientes requisitos funcionales son los que se han identificado en el proyecto:

- Manejo del catálogo con un diseño de CRUD y un sistema capaz de registrar, consultar, actualizar y borrar productos con atributos como nombre, descripción y precio unitario.
- Sistema para generar códigos QR en archivos PNG o SVG de manera automática para que cada uno sea único para cada producto, unido a su respectivo ID.
- Registro en tiempo real de las transacciones con cada elemento del stock para capturar cualquier entrada, salida o transferencia interna.
- Sistema de alerta que genera notificaciones de manera automática para los administradores y trabajadores cuando el nivel de stock de un producto cae por debajo de un mínimo establecido.
- Control del acceso basado en los permisos de cada tipo de usuario (operador, administrador, auditor).

4.2.2. Requisitos no funcionales

Los requerimientos no funcionales son aquellos que se encargan de definir las características de calidad y las limitaciones técnicas. Se han detectado los siguientes requerimientos no funcionales:

- La interfaz desarrollada con React debe ser responsive para funcionar de manera óptima en dispositivos móviles y tablets, permitiendo al usuario acceder a ella desde cualquier lugar del almacén.

- El sistema de reconocimiento de códigos QR y de actualización del stock debe ser inmediato, minimizando la latencia para evitar ralentizar el flujo logístico.
- La base de datos debe estar en MySQL con seguridad de datos garantizada. Las contraseñas de cada usuario deben estar codificadas con técnicas hash.
- El sistema debe ser capaz de operar en hardware de bajo coste para evitar un gasto excesivo en hardware especializado como escáneres industriales.
- La automatización del sistema de transacciones debería poder reducir la tasa de error hasta un máximo del 1%, asegurando precisión e integridad.

4.3 Actores

Para que el sistema opere de manera segura y organizada deben existir 3 actores principales con ciertas limitaciones en cuanto a cómo interactúan con la base de datos. Esta distinción es esencial para mantener un registro firme y ordenado de todos los activos de la compañía. Estos son los 3 principales actores:

- **Administrador:** Tiene control total sobre el sistema y sus funciones incluyen gestión, configuración del sistema ABC de clasificación de productos, supervisión de los reportes financieros y aprobación de los ajustes que sean necesarios en el sistema según la situación.
- **Operador:** El operador es el usuario que interactúa de manera directa con los productos físicos. Su perfil está orientado a la movilidad y sus tareas prioritarias son el escaneo de códigos QR para registrar cualquier tipo de transferencia y la confirmación de la recepción del stock.
- **Auditor:** Este perfil está enfocado en la consulta y tienen permiso para visualizar el historial de movimientos y transacciones; y los niveles de stock en tiempo real para generar reportes de rendimiento. Su objetivo en esencia es detectar fallos o discrepancias y verificar la integridad de los procesos aunque no tengan la capacidad de realizar movimientos o modificar los productos disponibles.

5 Diseño y navegación

5.1 Prototipos

El diseño de la interfaz está centrado en la usabilidad desde un dispositivo móvil y la eficiencia en un entorno industrial. Ya que el sistema debe ser capaz de operar en dispositivos de bajo coste, se usará React para crear una interfaz responsive que se pueda usar desde cualquier dispositivo móvil.

Algunos de los aspectos clave de la futura página son los siguientes:

- Dashboard principal: actúa como un centro de control para el administrador desde el cual se pueden visualizar alertas y los valores de stock de todo el sistema. En el caso de un operador, priorizaría el acceso a la interfaz de escáner para los códigos QR.
- Interfaz de escáner de códigos QR: está diseñada para la interacción inmediata y permite actualizaciones de los valores de stock en tiempo real sin latencia perceptible.
- Manejo de catálogo: diseño CRUD para visualizar, crear o editar productos con validación en tiempo real de campos requeridos vacíos o valores incorrectos, proporcionando un feedback visual instantáneo al usuario.
- Visualizador de detalle: presenta información del producto de manera condensada y con priorización en algunos datos, incluyendo la categoría ABC y un historial de movimientos que se sincroniza en tiempo real con la base de datos de SQL.

Todo el diseño de la interfaz sigue un patrón de jerarquía visual clara en el que los elementos más importantes (como el indicador de alerta) tienen el mayor contraste para ser fácilmente visibles. Existen interacciones pequeñas para crear feedback al ocurrir una operación, informando al usuario sobre el estado de éxito o error de dicha operación.

5.2 Guía de Estilo

El estilo visual de la página se define con una guía de estilo llamada “Soft Industrial”, la cual pretende crear una relación entre las herramientas digitales y el entorno industrial de un almacén, aportando una sensación de robustez a la interfaz.

Paleta de colores

Los colores empleados deben ser capaces no sólo de aportar cierta personalidad a la interfaz, sino que también deben ser capaces de aportar información de manera inmediata. Estos colores se han elegido para crear una interfaz con alto contraste, cumpliendo con los estándares WCAG 2.1 AA para asegurar legibilidad en diferentes condiciones de luz.

Color	Código HEX	Uso
Midnight Ink	#1A192A	Barras de navegación
Structured Concrete	#8B9097	Cartas y paneles secundarios
Safety Gold	#F2A900	Alertas y avisos críticos
Hyperlink Blue	#1D6A92	Botones de acción y links interactivos
Logistics Green	#00BF7D	Confirmaciones y avisos de éxito
Scorched Red	#C92306	Errores en el escaneo o discrepancias

Tipografía

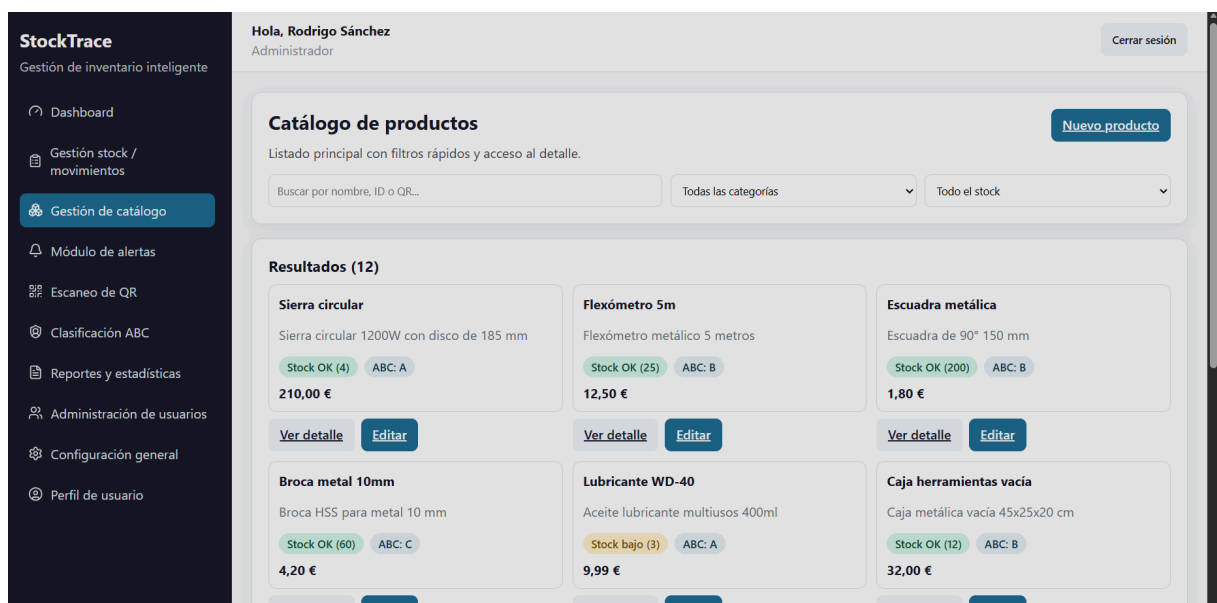
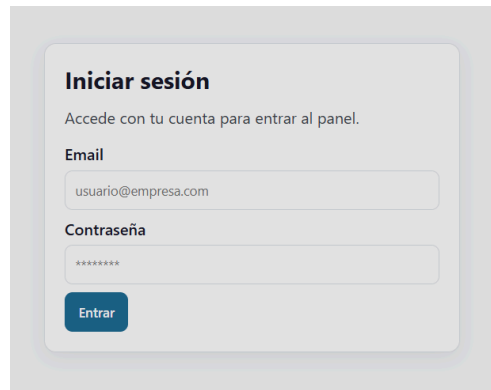
La fuente “Inter” se ha elegido como principal para la mayoría de textos de la interfaz. El tamaño base para la letra se establece en 16px con un espacio amplio entre cada línea para facilitar la legibilidad en pantallas más pequeñas.

Iconografía

La iconografía está basada en la librería Lucide-React, seleccionando símbolos según el nombre directo en cada caso. Todos los componentes siguen el mismo diseño para ser reusados múltiples veces en toda la página.

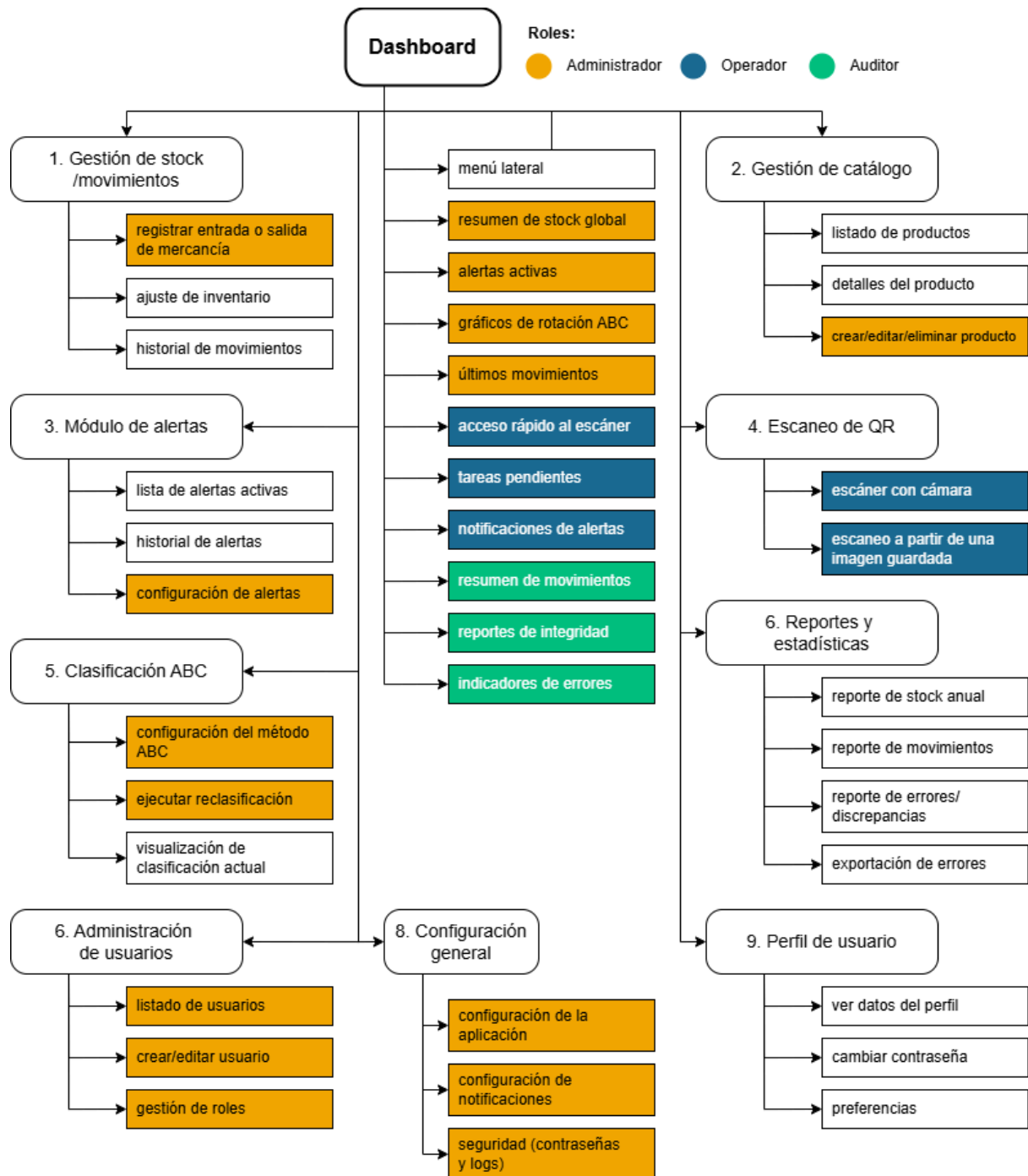
Imágenes

A continuación se presentan varias imágenes de la web en funcionamiento usando todos los elementos y pautas de diseño establecidos en este apartado.



5.3 Mapa de navegación

A continuación se presenta un mapa de navegación para la aplicación de gestión de inventario con códigos QR.

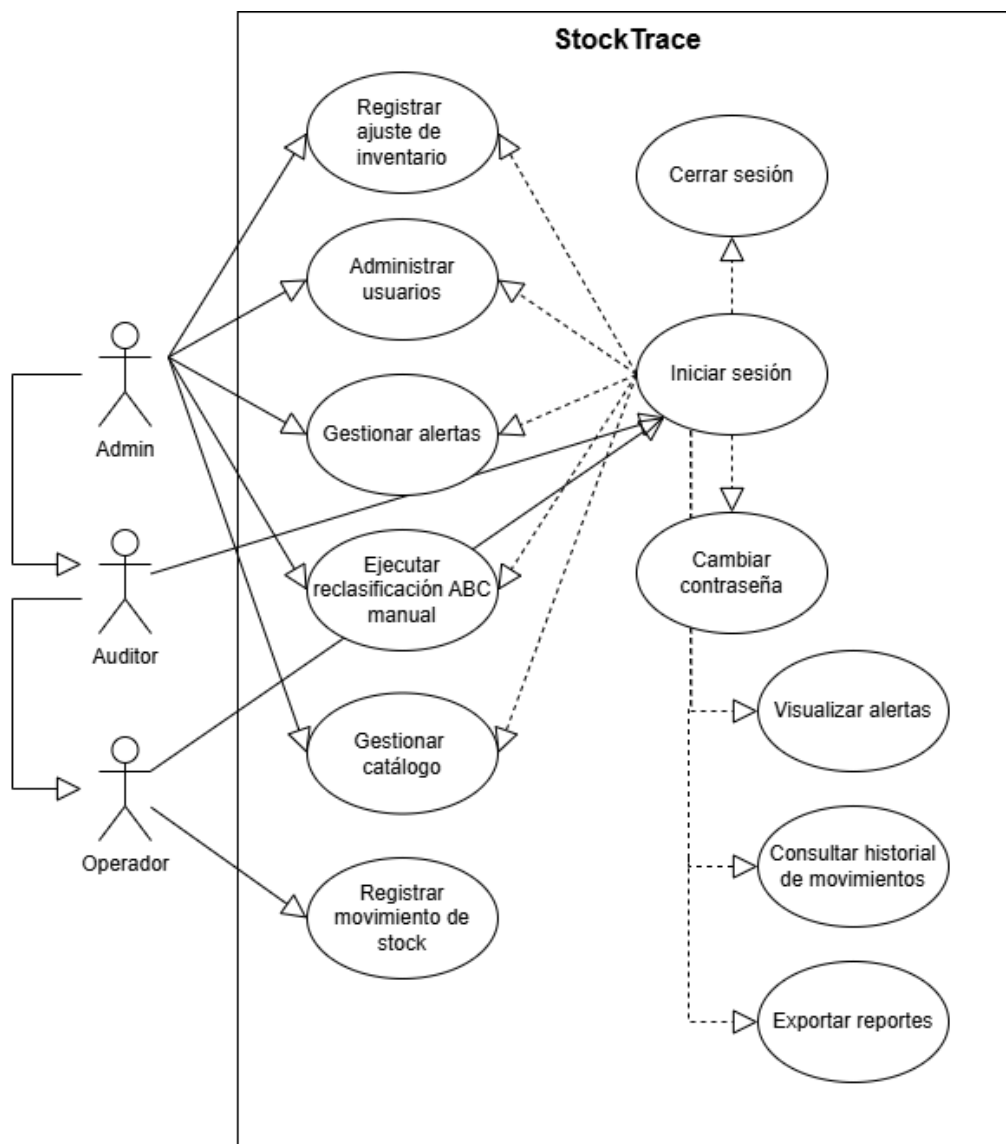


Se han tenido en cuenta los tres tipos de usuario (administrador, operador y auditor) de manera que cada uno disponga de una accesibilidad óptima según las funciones de cada rol. El mapa incluye todas las pantallas, subpantallas y acciones posibles. Todas las funciones han sido catalogadas según los roles que puedan llevar a cabo dicha acción. Ya

que el sistema de roles utiliza un sistema de jerarquía, todo lo que puede hacer un auditor lo puede hacer un operador y todo lo que puede hacer cualquiera de los dos, también lo puede hacer el administrador. Con este sistema de jerarquía se eliminan posibles dependencias entre los empleados de cada rol distinto y cada uno está limitado a un set de acciones que puede hacer por su cuenta.

6 Diagrama de Casos de Uso

6.1 Diagrama de Casos de Uso



6.2 Tablas de Especificación

Caso 1: Gestión de catálogo

Objetivo	Crear, leer, actualizar o eliminar registros de productos.
Actor	Administrador.
Precondiciones	El usuario debe estar autenticado por el administrador.
Flujo Principal	1. El Administrador accede al módulo de Catálogo. 2. Selecciona la acción deseada (Crear/Editar). 3. Completa los campos: nombre, descripción, precio, categoría ABC y stock mínimo. 4. El sistema valida los datos y actualiza la BD.
Flujos Alternativos	4-a. Datos obligatorios incompletos: El sistema muestra mensaje "Debe completar todos los campos". 4-b. ID de producto ya existe: El sistema informa que el código ya está registrado. 4-c. Valor numérico inválido: El sistema rechaza precios o cantidades negativas.
Postcondiciones	La información del producto se actualiza en tiempo real para todos los usuarios.

Caso 2: Registro de Entrada de Mercancía

Objetivo	Incrementar las existencias de un producto.
Actor	Operador.
Precondiciones	El producto debe estar previamente registrado en el catálogo.
Flujo Principal	1. El Operador abre el escáner en su smartphone. 2. Escanea el código QR del producto recibido. 3. El sistema identifica el artículo y muestra su estado actual. 4. El Operador ingresa la cantidad recibida. 5. El sistema actualiza el stock y registra la transacción con firma del Operador.
Flujos Alternativos	2-a. El QR es ilegible: El Operador busca el producto manualmente por nombre o ID. 4-a. Cantidad errónea: El sistema solicita confirmación si la entrada supera un umbral inusual.
Postcondiciones	La información del producto se actualiza en tiempo real para todos los usuarios.

Caso 3: Sistema de Alertas de Stock Mínimo

Objetivo	Notificar automáticamente cuando un producto requiere reposición.
Actor	Sistema (Automático).
Precondiciones	El producto tiene configurado un nivel de stock mínimo (Punto de Reorden).
Flujo Principal	1. Tras una salida de stock, el sistema recalcula el balance. 2. Si Stock actual < Stock mínimo, se activa un evento de alerta. 3. El sistema envía notificaciones al Administrador vía dashboard y email. 4. Se genera una sugerencia de pedido al proveedor habitual.
Flujos Alternativos	3-a. Error en envío de email: La alerta queda registrada de forma persistente en el panel de control para visualización posterior.
Postcondiciones	La información del producto se actualiza en tiempo real para todos los usuarios.

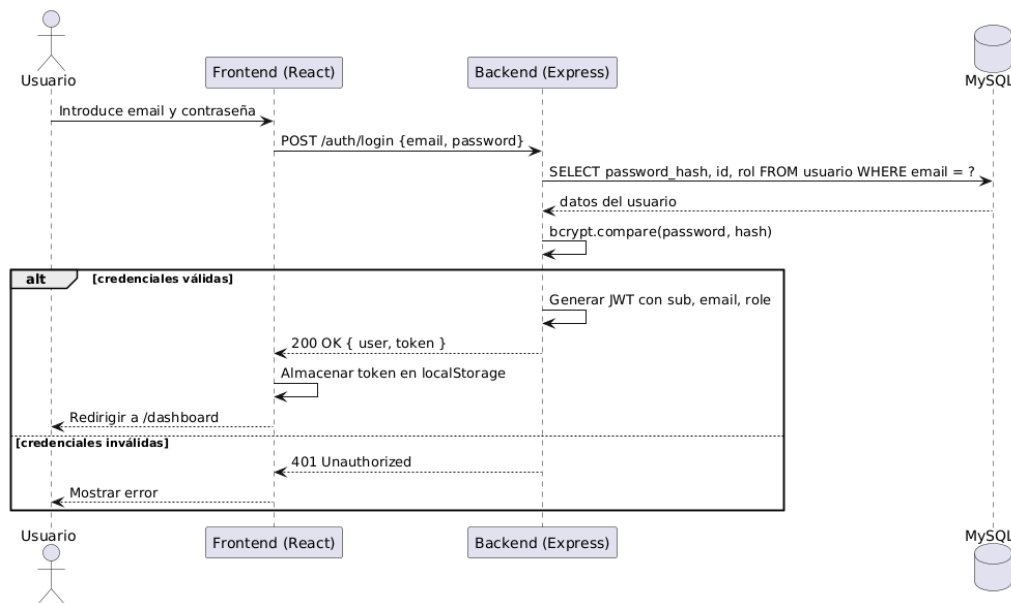
Caso 4: Reclasificación Dinámica ABC

Objetivo	Actualizar las categorías A, B o C basadas en la rotación e ingresos recientes.
Actor	Sistema / Administrador.
Precondiciones	Existencia de registros históricos de movimientos de al menos 30 días.
Flujo Principal	1. El sistema analiza los movimientos del periodo configurado. 2. Calcula la contribución de cada producto a los ingresos. 3. Aplica los umbrales de Pareto (80/20). 4. Actualiza las categorías en la base de datos.
Flujos Alternativos	3-a. Datos insuficientes para el periodo: El sistema mantiene la clasificación anterior e informa al Administrador. 4-a. Ajuste manual: El Administrador puede forzar la categoría de un producto crítico independientemente del cálculo.
Postcondiciones	Categorías actualizadas en BD, generación de reporte de cambios y sugerencias de reubicación física.

7 Diagramas de Secuencia

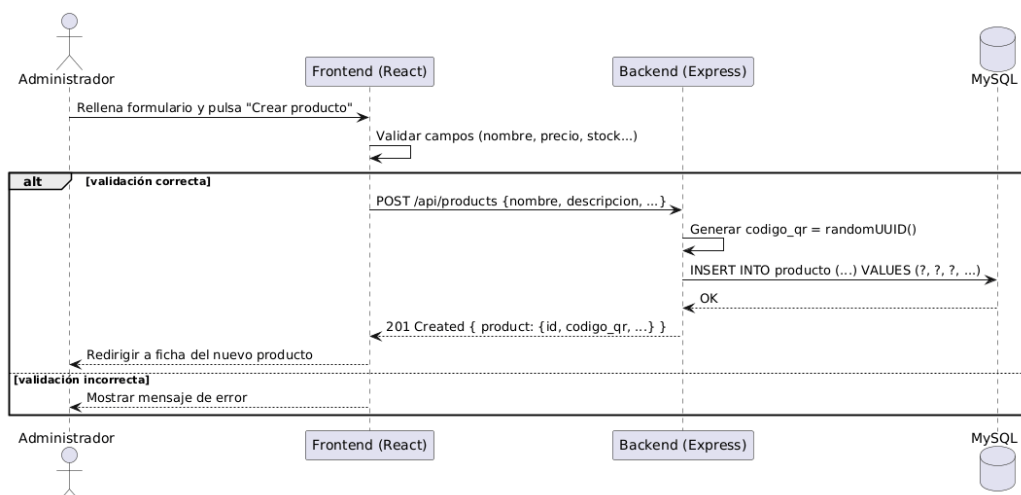
7.1 Inicio de sesión (Login)

Este diagrama muestra el intercambio de mensajes cuando un usuario introduce sus credenciales para autenticarse. El frontend envía la petición al endpoint `/auth/login`, el backend valida la contraseña cifrada con `bcrypt` y devuelve un token JWT que se almacena en el cliente.



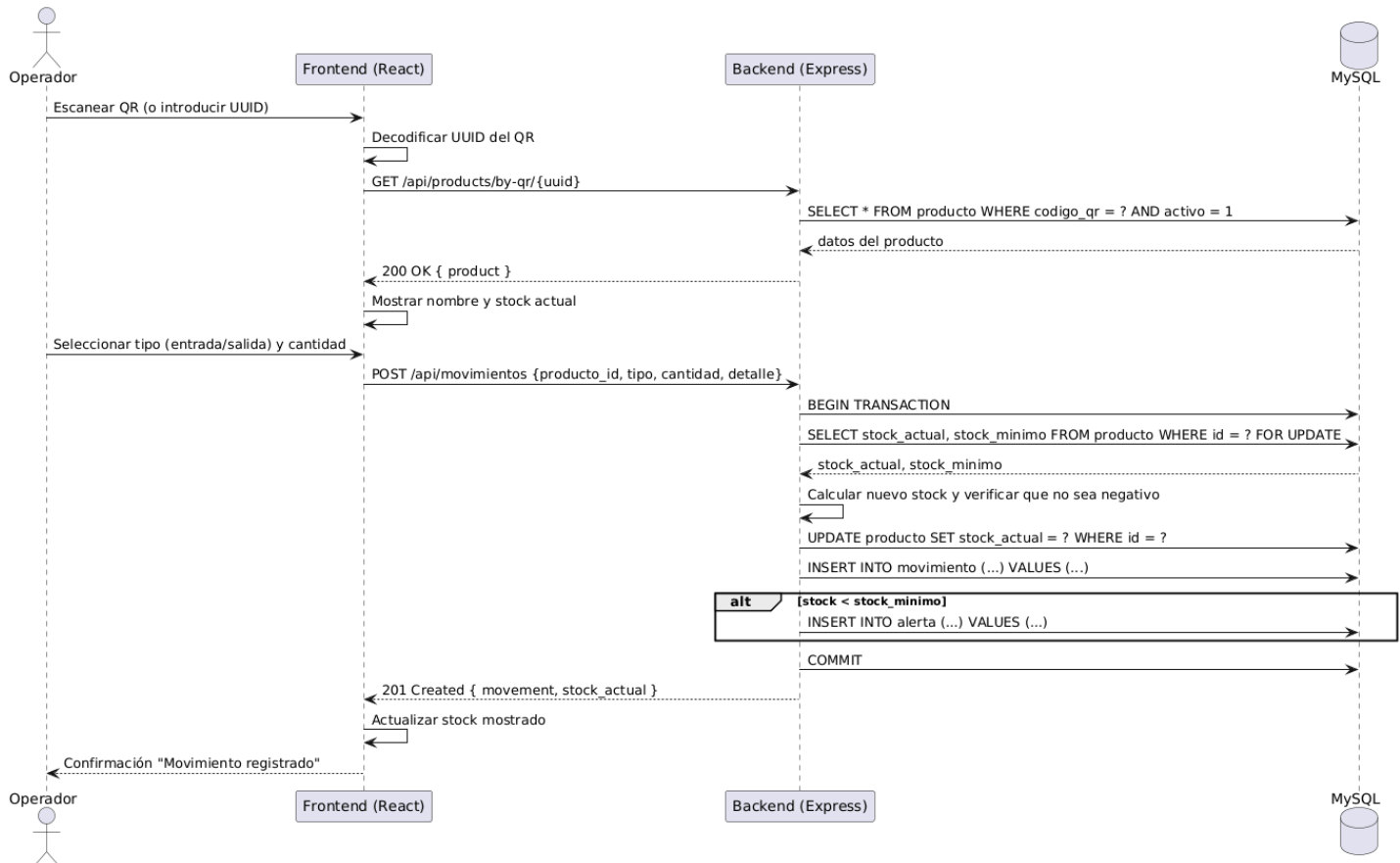
7.2 Creación de un producto (CRUD catálogo)

Secuencia para la creación de un nuevo producto por parte del administrador. El frontend envía los datos validados, el backend genera un UUID para el código QR y almacena el registro.



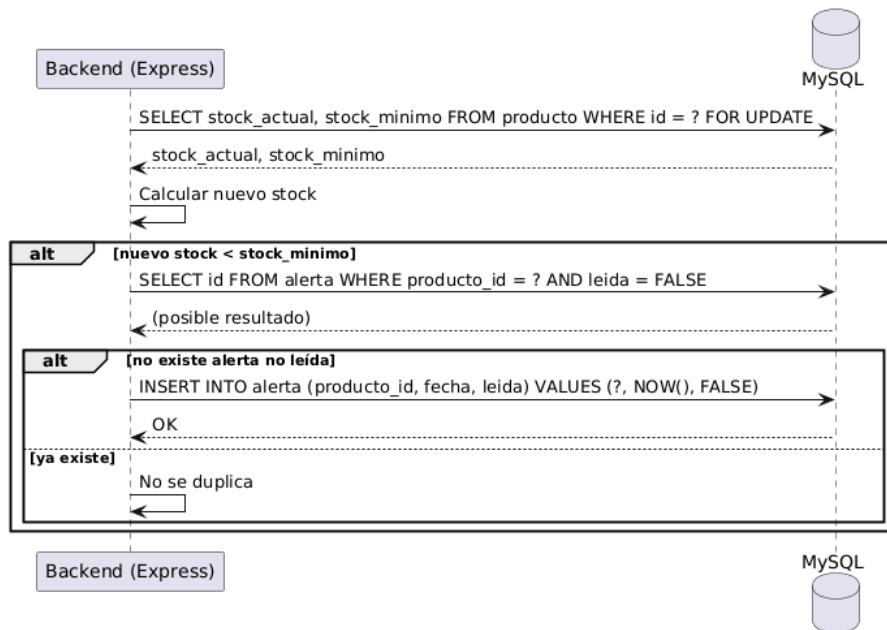
7.3 Registro de movimiento mediante escaneo QR

Este es el caso de uso central del sistema. El operador escanea un código QR que contiene el UUID del producto, el frontend consulta el producto, el operador introduce la cantidad y el tipo de movimiento, y el backend actualiza el stock dentro de una transacción.



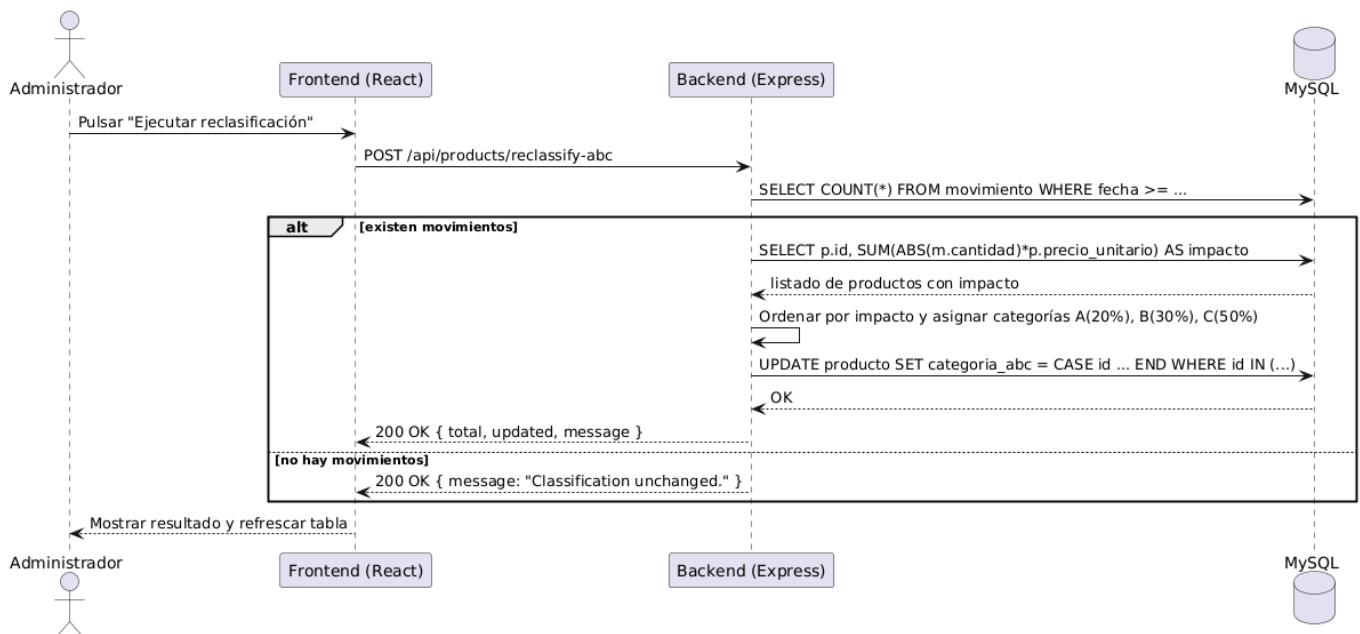
7.4 Generación automática de alertas (disparada por un movimiento)

Este diagrama detalla el subflujo de creación de alertas que se ejecuta dentro de la transacción del movimiento, cuando el nuevo stock queda por debajo del mínimo configurado. Es una ampliación del paso condicional del diagrama anterior.



7.5 Reclasificación ABC manual

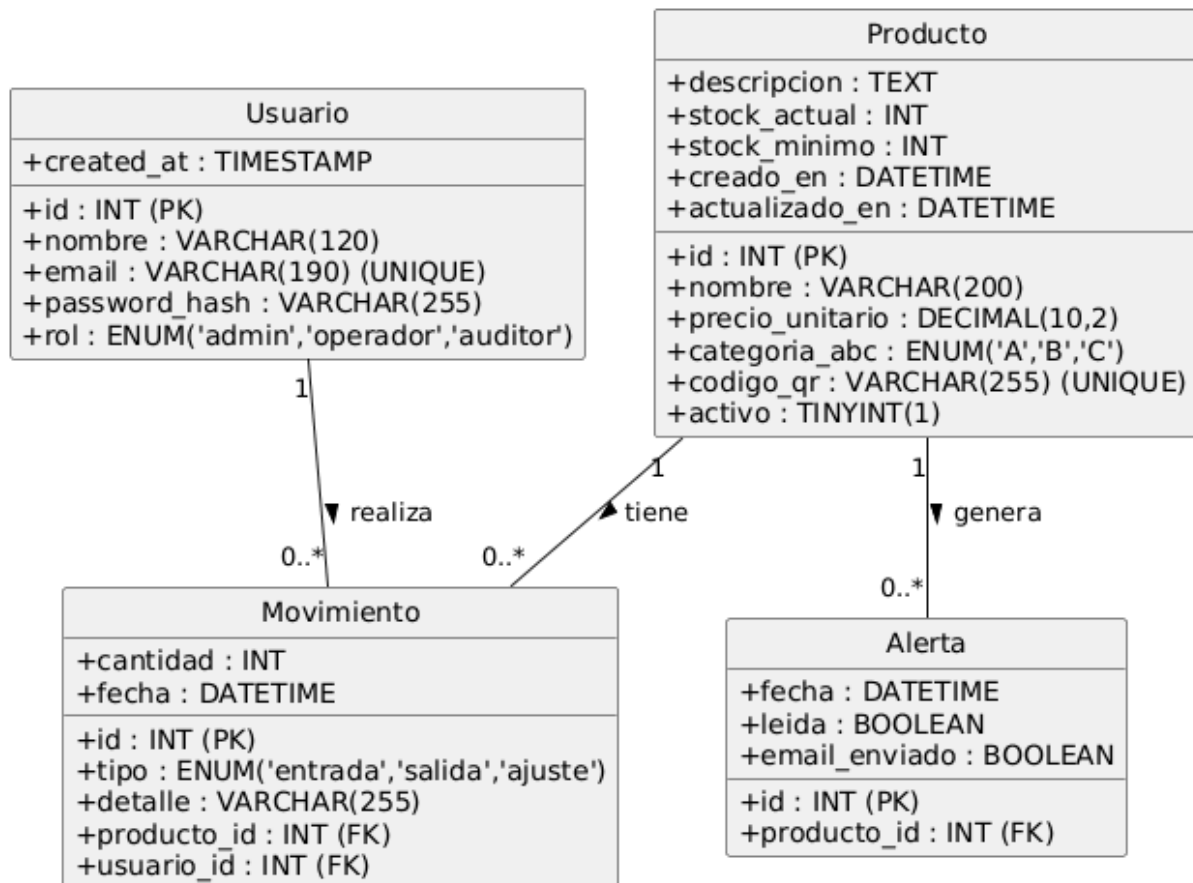
El administrador puede forzar la reclasificación ABC desde la página correspondiente. La petición llega al backend, que ejecuta el algoritmo de Pareto sobre los movimientos de los últimos 30 días y actualiza las categorías en bloque.



8 Diagramas de Clase

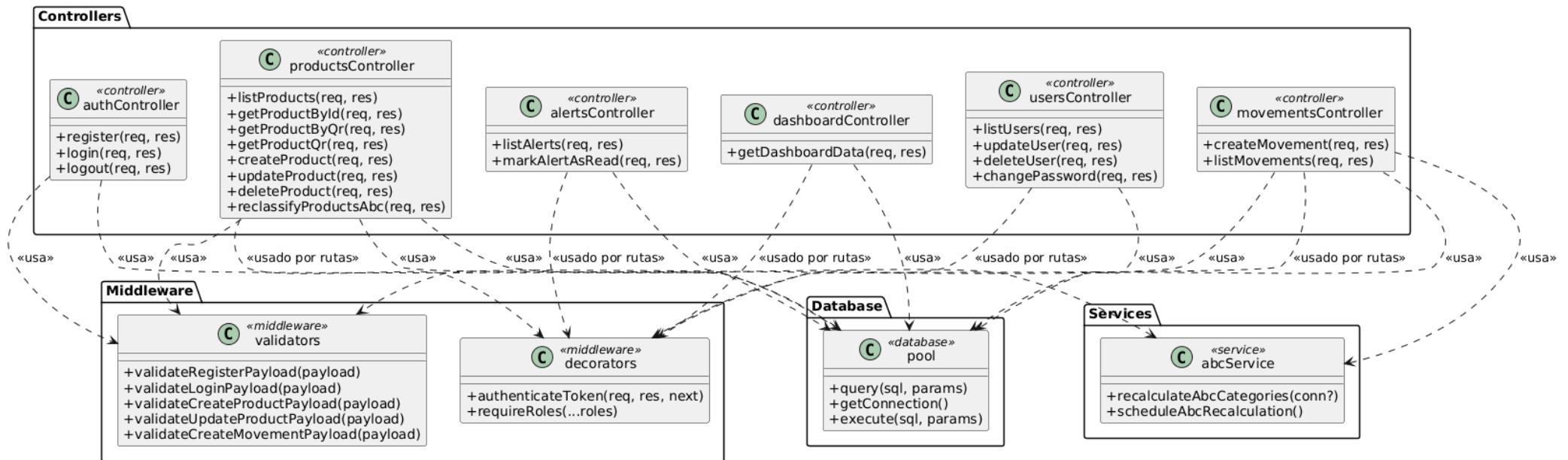
8.1 Diagrama de clases del modelo de datos

Este diagrama representa cada tabla de MySQL como una clase UML. Los atributos se corresponden con las columnas definidas en el script de creación de la base de datos. Las relaciones muestran cómo las claves foráneas conectan las entidades.

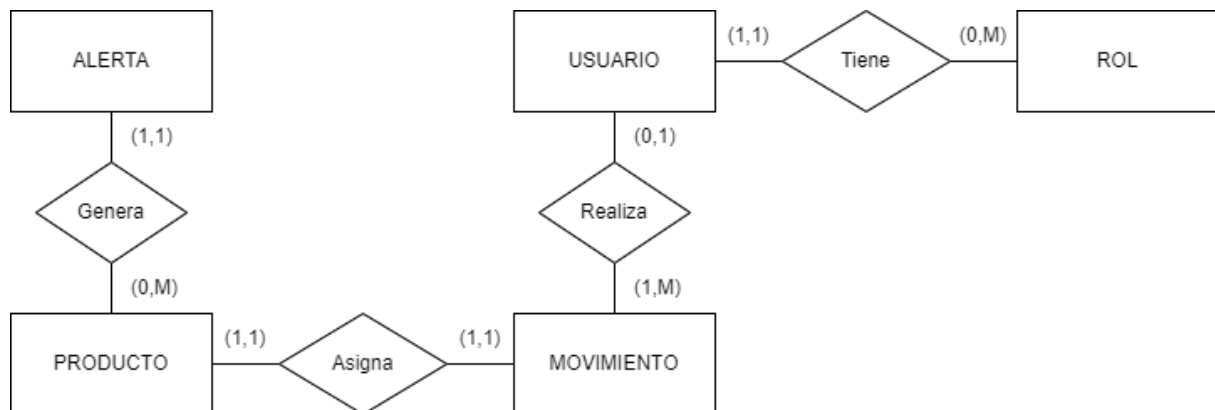


8.2 Diagrama de clases de diseño (arquitectura del backend)

La arquitectura del servidor Express se estructura en módulos independientes. Cada uno de estos módulos exporta funciones (métodos estáticos en el diagrama) que encapsulan la lógica correspondiente. La siguiente representación ayuda a visualizar las responsabilidades y la comunicación entre capas.



9 Modelo Entidad-Relación



Tablas de la Base de Datos

Tabla de USUARIO

Campo	Tipo	Descripción
id	INT PK	Identificador único
nombre	VARCHAR(100)	Nombre completo
email	VARCHAR(100)	Correo electrónico (único)
password_hash	VARCHAR(255)	Contraseña cifrada
id_rol	INT FK	Referencia al identificador del rol
fecha	DATETIME	Fecha de registro

Tabla de ALERTA

Campo	Tipo	Descripción
id	INT PK	Identificador único
producto_id	INT FK	Producto con stock bajo
fecha	DATETIME	Fecha de generación
leida	BOOLEAN	Indica si el administrador ya la revisó
email_enviado	BOOLEAN	Control para no reenviar

Tabla de PRODUCTO

Campo	Tipo	Descripción
id	INT PK	Identificador único
nombre	VARCHAR(200)	Nombre del producto
descripcion	TEXT	Descripción detallada
precio_unitario	DECIMAL(10,2)	Precio por unidad
stock_actual	INT	Cantidad disponible
stock_minimo	INT	Nivel para activar alerta
categoria_abc	ENUM('A','B','C')	Clasificación según método ABC
codigo_qr	VARCHAR(255)	Identificador único para el QR (ej. UUID)
creado_en	DATETIME	Fecha de alta
actualizado_en	DATETIME	Última modificación

Tabla de MOVIMIENTO

Campo	Tipo	Descripción
id	INT PK	Identificador único
producto_id	INT FK	Producto afectado
tipo	ENUM	'entrada', 'salida', 'ajuste'
cantidad	INT	Número de unidades (positivo para entradas, negativo para salidas)
fecha	DATETIME	Momento de la transacción
usuario_id	INT FK	Operador que realizó el movimiento
detalle	VARCHAR(255)	Observaciones (ej. "recepción de proveedor")

Tabla de ROL

Campo	Tipo	Descripción
id	INT PK	Identificador único
nombre	VARCHAR(50)	Nombre del rol (administrador, operador, auditor)
descripcion	VARCHAR(255)	Explicación de las funciones del rol
nivel	INT	Opcional: jerarquía (1=admin, 2=operador, 3=auditor)

10 Marketing

10.1 Análisis de Macroentorno y Microentorno

El análisis del entorno es fundamental para cualquier proyecto, ya que permite identificar y comprender los factores que pueden influir en su éxito o fracaso. En el caso del desarrollo de una aplicación web, realizar un análisis detallado del macroentorno y microentorno ayuda a detectar oportunidades y amenazas externas, así como fortalezas y debilidades internas. A continuación, se explica cada uno de estos análisis y cómo aplicarlos en el contexto de un proyecto de desarrollo de aplicaciones web.

10.1.1. Análisis del Microentorno

El microentorno se refiere a los factores que afectan directamente a la empresa o proyecto y que están más cerca de su área de influencia directa. Estos factores son específicos de la industria o sector en el que opera el proyecto, en este caso, el desarrollo de aplicaciones web. En este análisis, se deben considerar los siguientes elementos clave:

a) Competencia

Es fundamental identificar a los competidores directos e indirectos. ¿Qué productos o servicios ofrecen y cómo se diferencian del proyecto en desarrollo? ¿Cuál es su cuota de mercado y qué estrategias emplean para atraer usuarios? Es importante analizar también las barreras de entrada en el mercado y si existe alguna ventaja competitiva que pueda aprovecharse.

b) Clientes

Se debe investigar quiénes son los clientes potenciales o usuarios de la aplicación. ¿Cuál es su perfil demográfico, sus intereses, hábitos de consumo y necesidades? ¿Cómo suelen interactuar con aplicaciones web similares? Asignatura: PDAM Entrega 2 Entender las expectativas y preferencias de los clientes es clave para diseñar y desarrollar una aplicación que cumpla con sus demandas y se posicione de manera efectiva en el mercado.

c) Proveedores y Colaboradores

Los proveedores pueden incluir servicios de hosting, APIs, herramientas de software o empresas de infraestructura tecnológica que sean esenciales para el funcionamiento de la aplicación. Evaluar la relación con estos proveedores y colaboradores es crucial para garantizar la sostenibilidad y escalabilidad del proyecto.

d) Intermediarios

En el caso de aplicaciones web, los intermediarios pueden ser plataformas de distribución (como las tiendas de aplicaciones), socios tecnológicos o plataformas de marketing que ayudan a promover y distribuir el producto. Analizar cómo estos intermediarios influyen en el éxito del proyecto y si hay alternativas que permitan optimizar la cadena de valor. El análisis del microentorno se centra en factores específicos y cercanos que afectan directamente la operatividad del proyecto y son, en gran medida, controlables por la empresa. Permite comprender mejor las fortalezas y debilidades del proyecto en su contexto inmediato.

10.1.2. Análisis del Macroentorno

El macroentorno se refiere a los factores externos que afectan a la empresa o proyecto de manera indirecta y que están fuera de su control directo. El análisis PESTEL es una herramienta útil para evaluar estos factores, ya que se enfoca en los aspectos Políticos, Económicos, Sociales, Tecnológicos, Ecológicos y Legales.

Cada uno de ellos se describe a continuación:

a. Político

Incluye regulaciones gubernamentales, políticas fiscales y comerciales, y leyes que afectan al sector tecnológico y de desarrollo web. Es crucial entender cómo estas decisiones pueden influir en el proyecto. Por ejemplo, políticas de privacidad de datos como el GDPR en la UE impactan directamente en cómo se gestionan y almacenan los datos de los usuarios.

b. Económico

Factores como el crecimiento económico, la inflación, las tasas de interés y el poder adquisitivo del público objetivo son elementos importantes en esta categoría. Un entorno económico estable y favorable puede estimular la demanda de aplicaciones y servicios digitales, mientras que una crisis económica podría limitar el alcance del proyecto.

c. Social

Los factores sociales abarcan cambios demográficos, tendencias culturales, comportamiento del consumidor y actitudes hacia la tecnología. Por ejemplo, si el público objetivo está compuesto por jóvenes, es importante que la aplicación sea compatible con dispositivos móviles y tenga una experiencia de usuario adaptada a este segmento.

d. Tecnológico

Este aspecto analiza las innovaciones tecnológicas, la infraestructura tecnológica disponible y las tendencias del sector. Evaluar la velocidad de adopción de nuevas tecnologías, la disponibilidad de herramientas de desarrollo o la conectividad de Internet es clave para determinar si el proyecto es viable y si se puede integrar con las tecnologías emergentes.

e. Ecológico (Ambiental)

Aunque no siempre se considera prioritario en el desarrollo de software, el factor ecológico puede ser relevante si el proyecto se propone tener una huella ambiental baja. Un ejemplo sería utilizar proveedores de hosting que se comprometan con la sostenibilidad o que utilicen energías renovables.

f. Legal

Se refiere a las normativas y leyes que rigen el mercado en el que se va a operar. Esto incluye desde leyes de propiedad intelectual hasta regulaciones específicas del sector tecnológico, como las de comercio electrónico o privacidad de datos. Cumplir con estas regulaciones es crucial para evitar sanciones y garantizar la legalidad del proyecto.

10.2 DAFO

El siguiente análisis DAFO tiene como objetivo el identificar todos los factores internos y externos que puedan afectar a la viabilidad del proyecto, aportando una visión estratégica para un desarrollo adecuado.

10.2.1. Factores internos

Fortalezas

Las fortalezas son factores internos positivos que suponen ventajas competitivas. Se han identificado las siguientes fortalezas:

- Automatización de la gestión del inventario.

El sistema elimina la necesidad de manejar datos de manera manual, reduciendo así de manera drástica el margen de error más común en aquellos sistemas que no estén automatizados (menos de 1% de error como objetivo).

- Implementación de bajo coste.

La manera en la que está planteado el proyecto elimina desde un inicio la necesidad de usar cualquier tipo de hardware especializado que suelen ser requeridos para el manejo de stock. Esto reduce la barrera de entrada de las PYMEs significativamente ya que ya no es necesaria una inversión inicial en hardware.

- Integración nativa del sistema de priorización ABC.

El método ABC para clasificar productos basado en su valor está presente desde la fase más inicial de diseño del proyecto, optando por un enfoque en la optimización a la hora de manejar recursos y controlar el stock.

- Tecnología actualizada y de código abierto.

El proyecto usa Node.js (Express), React y MySQL; tecnologías bien establecidas, con grandes comunidades y sin costes de licencia.

- Trazabilidad completa y con visibilidad en tiempo real.

Cada producto en la base de datos posee su propio código QR con información detallada de todos sus movimientos que es registrada el mismo instante en el que se realiza. Los administradores reciben notificaciones y alertas en tiempo real para reaccionar lo más rápido posible ante cualquier fallo en el sistema.

- Escalabilidad y adaptabilidad.

La arquitectura de React basada en el uso de componentes y el uso de la REST API permite un amplio crecimiento en función de los requerimientos sin necesidad de una reestructuración interna.

- Enfoque en la experiencia del usuario móvil.

La interfaz está diseñada para un uso diario y sencillo en dispositivos móviles, facilitando así la adopción por parte de los empleados del almacén.

Debilidades

Las debilidades son factores internos positivos que suponen desventajas frente a la gran mayoría de competidores. Se han identificado las siguientes debilidades:

- Proyecto en fase conceptual.

En el momento actual, el proyecto se encuentra en fase preliminar o de validación sin usuarios reales, ya que el desarrollo todavía no ha empezado.

- Posible curva de aprendizaje.

Muchos empleados dentro de las PYMEs, aunque se pretenda minimizar lo máximo posible, aún así pueden verse desafiados por la complejidad del sistema, sobre todo en el caso de que ya estén acostumbrados a un sistema más tradicional de gestión de inventario.

- Retrocompatibilidad con sistemas tradicionales.

Aunque no está decidido completamente, es posible que crear cierto nivel de compatibilidad con los sistemas de gestión tradicionales se aleje demasiado del enfoque principal del proyecto y añada demasiada carga a un proyecto tan pequeño. Por ello, es muy probable que no se llegue a desarrollar esta compatibilidad.

- Mantenimiento y soporte post-lanzamiento.

No se ha planeado ningún sistema de mantenimiento post-lanzamiento todavía para el proyecto, lo cual es algo esencial para mantener la confianza con el cliente.

- Enfoque limitado a la gestión de inventario.

El proyecto solo comprende la gestión de un inventario a través de una base de datos; y no comprende el manejo de las compras de los clientes, las ventas, las cuentas... Lo cual podría forzar al cliente a usar varias herramientas al mismo tiempo.

2.2. Factores externos

Oportunidades

Las oportunidades son factores externos positivos que comprenden momentos específicos o cualidades que permiten un desarrollo más eficiente en comparación a una situación normal. Se han identificado las siguientes oportunidades:

- Aumento en la digitalización de las PYMEs.

En los últimos años, las PYMEs buscan modernizar sus procesos por medio de la digitalización.

- Tendencia al trabajo en remoto y el uso de la nube para manejar datos.

Se prefieren soluciones basadas en la web por encima de instalaciones locales.

- Evolución del uso de códigos QR.

Hoy en día los códigos QR están muy normalizados y se usan en multitud de situaciones.

Amenazas

- Gran competencia ya establecida.

Ya existen numerosos modelos de gestión de inventario con costes muy bajos o incluso gratis.

- Seguridad de datos y privacidad.

Muchas empresas pueden rechazar este tipo de proyecto por el miedo a almacenar datos sensibles e importantes en la nube si no se garantizan estándares de protección.

- Dependencia en conexión a Internet.

Aunque el sistema pueda estar diseñado para operar sin conexión, aún no se ha especificado esta propiedad.

- Inestabilidad económica.

Un periodo de crisis o recesión puede ralentizar los procesos de inversión de dinero por parte de las empresas hacia este tipo de proyectos.

DEBILIDADES • Proyecto en fase conceptual. • Posible curva de aprendizaje. • Retrocompatibilidad con sistemas tradicionales. • Mantenimiento y soporte post-lanzamiento. • Enfoque limitado a la gestión de inventario.	AMENAZAS • Gran competencia ya establecida. • Seguridad de datos y privacidad. • Dependencia en conexión a Internet. • Inestabilidad económica.
FORTALEZAS • Automatización de la gestión del inventario. • Implementación de bajo coste. • Integración nativa del sistema de priorización ABC. • Tecnología actualizada y de código abierto. • Trazabilidad completa y con visibilidad en tiempo real. • Escalabilidad y adaptabilidad. • Enfoque en la experiencia del usuario móvil.	OPORTUNIDADES • Aumento en la digitalización de las PYMEs. • Tendencia al trabajo en remoto y el uso de la nube para manejar datos. • Evolución del uso de códigos QR.

10.3 Plan de Marketing

El plan de marketing de StockTrace se articula en torno al objetivo de posicionar la plataforma como la opción de referencia para la digitalización del inventario en pequeñas y medianas empresas, aprovechando las ventajas competitivas identificadas en el análisis DAFO: automatización, bajo coste de implantación y ausencia de dependencia de hardware especializado.

10.3.1 Público objetivo y segmentación

El mercado potencial se segmenta en tres perfiles principales, ordenados por prioridad:

Segmento	Descripción	Necesidades clave
PYMEs con almacén físico	Empresas de distribución, talleres, comercios mayoristas y minoristas con entre 5 y 50 empleados.	Sustituir el registro manual (hojas de cálculo o papel) por un sistema digital asequible, sin invertir en escáneres ni infraestructura costosa.
Startups y proyectos de e-commerce	Negocios digitales que necesitan controlar el inventario de forma ágil desde su creación.	Una herramienta que escale con su crecimiento, accesible desde cualquier dispositivo y que no requiera formación técnica.
Pequeñas empresas de logística y almacenaje	Operadores que gestionan stock de terceros y necesitan trazabilidad y reportes para sus clientes.	Trazabilidad completa, control de accesos por roles y capacidad de exportar datos para auditoría.

10.3.2 Estrategia de producto

StockTrace se presenta como una solución Software as a Service (SaaS), accesible desde cualquier navegador moderno, sin necesidad de instalación local. La estrategia de producto se basa en cuatro pilares:

1. Facilidad de adopción: la interfaz está diseñada para funcionar en dispositivos móviles comunes; el escaneo de códigos QR no requiere hardware adicional. Cualquier empleado con un smartphone puede operar con el sistema tras una breve sesión de formación.

2. Código abierto y personalización: el proyecto se desarrolla íntegramente con tecnologías gratuitas (Node.js, React, MySQL), lo que elimina los costes de licencias y permite a empresas con departamento técnico propio adaptar o ampliar las funcionalidades.
3. Modelo de suscripción asequible: se plantean tres planes tarifarios, alineados con la reducida capacidad de inversión de las PYMEs (véase apartado 10.4). La suscripción mensual incluye alojamiento en la nube, soporte técnico básico y actualizaciones periódicas.
4. Propuesta de valor diferencial: a diferencia de otros sistemas gratuitos, StockTrace integra de forma nativa la clasificación ABC (principio de Pareto) y el sistema de alertas por stock mínimo. Esto proporciona a las PYMEs un control estratégico del inventario que normalmente solo ofrecen soluciones empresariales mucho más costosas.

10.3.3 Estrategia de precio

La política de precios se ha definido teniendo en cuenta los costes totales del proyecto (37.548 € de desarrollo inicial, más los costes recurrentes de infraestructura y soporte) y la capacidad económica del segmento objetivo. Se proponen los siguientes niveles:

Plan	Usuarios incluidos	Productos máx.	Funcionalidades	Precio mensual
Básico	Hasta 3	500	Catálogo, movimientos, escáner QR, alertas por correo	20€
Profesional	Hasta 10	2000	Todo lo anterior + clasificación ABC, reportes CSV, dashboard avanzado	40€
Empresa	Ilimitados	Ilimitados	Todo lo anterior + administración de roles personalizados, soporte prioritario, personalización	80€

Se contempla un periodo de prueba gratuito de 30 días con todas las funcionalidades activas, como estrategia de captación. La fricción de entrada es nula: el alta no requiere tarjeta de crédito.

10.3.4 Estrategia de distribución

El canal de distribución es exclusivamente online, a través del sitio web del producto.

La estrategia de distribución se apoya en tres vías:

- Captación directa mediante contenido: publicación de artículos y tutoriales sobre gestión de inventario, metodología ABC y digitalización de PYMEs en blogs sectoriales, foros especializados y redes profesionales (LinkedIn).
- Marketplaces de software: presencia en directorios como Capterra, GetApp o Software Advice, donde las PYMEs buscan y comparan herramientas de gestión empresarial.
- Colaboraciones con asociaciones empresariales: acuerdos de difusión con asociaciones de comerciantes, cámaras de comercio y consultoras especializadas en transformación digital de PYMEs.

10.3.5 Estrategia de comunicación

La comunicación se orientará a construir confianza y demostrar la utilidad inmediata del producto. Se prevén las siguientes acciones:

1. Demostraciones y webinars: sesiones en directo donde se muestre el flujo completo de trabajo (creación de producto, generación de QR, escaneo, movimiento y alerta) en menos de 10 minutos.
2. Casos de uso reales: testimonios y ejemplos prácticos de PYMEs que hayan reducido su tasa de errores gracias a la automatización.
3. Presencia en redes profesionales: publicaciones periódicas en LinkedIn con datos sobre el coste del error manual en inventarios y cómo las herramientas digitales pueden mitigarlo.
4. Documentación y soporte: un centro de ayuda con preguntas frecuentes, manuales descargables y tutoriales en vídeo, disponible incluso para usuarios no registrados

El tono de comunicación será cercano y práctico, evitando tecnicismos innecesarios y destacando siempre la promesa central: gestión de inventario inteligente sin hardware costoso.

10.3.6 Indicadores y seguimiento

Para evaluar la efectividad del plan de marketing, se monitorizarán los siguientes indicadores desde el lanzamiento:

- Número de altas en el periodo de prueba gratuito.
- Tasa de conversión de prueba a plan de pago.
- Tasa de abandono (churn) mensual.
- Origen del tráfico (búsqueda orgánica, redes sociales, directorios).
- Número de menciones y reseñas en plataformas especializadas.

10.4 Valoración del proyecto

Esta valoración descompone los recursos necesarios para llevar el proyecto desde la fase inicial hasta la fase de lanzamiento.

10.4.1 Costes directos

Concepto	Detalles	Coste aprox.
Horas de desarrollo	17 semanas de trabajo	27000€
Diseños de UI	Prototipado de alta calidad	1500€
Licencias	Figma Profesional y herramientas de desarrollo	60€
Infraestructura en la nube	VPS profesional, SSL y base de datos de MySQL	120€
Dominios web	Registro y protección para el primer año	20€

10.4.2 Costes indirectos

Concepto	Detalles	Coste aprox.
Costes administrativos	Cuota de autónomos	1120€
Impuestos y seguros	Seguro de Responsabilidad Civil Profesional	520€
Servicios legales	Adecuación al RGPD/LOPDGDD y redacción de textos legales para la web/app	250€
Formación tecnológica	Actualización en normativas Verifactu y nuevas librerías de escaneo QR	500€

10.4.3 Margen de beneficio

- Suma de Costes Totales (Directos + Indirectos): 31.290 €
- Margen de Beneficio (20%): 6.258 €
- Valoración Total del Proyecto (PVP estimado): 37.548 €

El proyecto presenta una valoración de 37500€ para una implementación que cubriría 17 semanas de desarrollo.

El 86% de los costes están concentrados en el desarrollo, dejando clara la importancia de metodologías ágiles como Kanban para incrementar la productividad y evitar posibles aumentos de gastos.

11 Pruebas

En el desarrollo de aplicaciones web, las pruebas son una parte esencial del proceso. Permiten garantizar que el sistema funcione de forma correcta, estable y segura antes de su publicación.

A través de los tests, se pueden detectar errores en la lógica del código, problemas de integración entre módulos, fallos en la experiencia de usuario y vulnerabilidades de seguridad.

Para este proyecto realizaremos diferentes tipos de pruebas, tanto unitarias como funcionales, de integración o de validación de la interfaz, con el objetivo de cubrir el mayor número de escenarios posibles.

11.1 Test Unitarios

Los tests unitarios son los tests más básicos ya que se centran en analizar el comportamiento de cada componente individual y su nivel de funcionalidad. La arquitectura propuesta para el proyecto es la de Node.js con Express y es esencial asegurarse del funcionamiento adecuado de la lógica de la aplicación antes de permitirle interactuar con la base de datos o la interfaz de React.

En el backend, los tests son necesarios para comprobar la validez de los datos existentes o que no interfieran entre sí. Al usar el sistema de catalogación ABC, surge un escenario ideal en el que probar estos tests, comprobando que las reglas

sobre las que funciona este sistema de catalogación se están aplicando correctamente a todos los productos del almacén.

En cuanto al frontend, los tests unitarios tienen como objetivo validar los diferentes componentes de la interfaz que usan React, la aplicación correcta de los estilos según la paleta de colores “Soft Industrial” y que los cambios en la interfaz se apliquen bajo las circunstancias adecuadas.

11.2 Test de Integración

Los tests de integración sirven para comprobar la interacción correcta entre los distintos módulos o componentes del sistema, detectando errores en dicha interacción según el código. En este tipo de proyecto con tantos componentes, los tests de integración son esenciales para asegurar las interacciones.

Un ejemplo de integración fundamental puede ser el registro de entrada de mercancía: el trabajador escanea el QR con la cámara, el frontend procesa la imagen y extrae la ID, se envía una petición a la API del backend y se actualiza la base de datos. Todas estas interacciones deben ocurrir de manera asíncrona y correctamente para que no existan errores en el proceso.

La arquitectura escalable que usa el proyecto implica el uso de tests de integración cada vez que se amplíe la escala del proyecto.

11.3 Test Funcionales o de Características

Los tests funcionales comprueban que el software cumpla con los requisitos mínimos establecidos y que cada función del sistema lleve a cabo su tarea. Estos tests son esenciales para permitir que los usuarios del sistema sean capaces de interactuar con el sistema correctamente. Para este proyecto, los tests funcionales o de características implican la validación del CRUD del catálogo, el registro de movimientos en tiempo real y las alertas.

11.4 Test de Usabilidad

Los tests de usabilidad tienen como objetivo evaluar la facilidad con la que los usuarios pueden interactuar con la aplicación, intentando encontrar un diseño de interfaz intuitiva, accesible y agradable de usar en el proceso. En el contexto de un almacén industrial, la accesibilidad y la eficiencia son piezas esenciales para

garantizar una operatividad óptima. Estos tests se centrarán sobre todo en la usabilidad móvil, ya que será la más importante para los trabajadores.

Estos tests comprueban cosas como la jerarquía visual de manera que los elementos más importantes destaquen por encima de otros y la accesibilidad.

11.5 Test de Seguridad

Los tests de seguridad comprueban que la aplicación maneje los datos de manera que estos estén protegidos y sea resistente a ataques o accesos no autorizados. Ya que en este proyecto se almacena información sobre partes esenciales de cómo funciona internamente una empresa, la seguridad es algo principal. Los tests deben alinearse con los estándares actuales de protección y seguridad, incluyendo el cifrado de contraseñas con técnicas hash.

Otro aspecto importante a tener en cuenta es el de los permisos por rol, se debe comprobar que cada usuario tenga una capacidad dentro del sistema de manera que se adecúe a su rol.

La seguridad de los datos también se extiende a los datos almacenados en la nube, cumpliendo con los estándares modernos de transmisión de datos para garantizar un nivel de seguridad adecuado.

11.6 Test de Rendimiento

Los tests de rendimiento son imprescindibles para garantizar un nivel de calidad adecuado. Estos tests analizan la velocidad, la estabilidad y el tiempo de respuesta del sistema bajo diversas circunstancias. En el ámbito de la gestión de almacenes, el más mínimo retraso a la hora de actualizar el stock puede dar lugar a errores en la cadena de suministro. Por ello, todo el sistema debe operar de manera inmediata y lo más eficientemente posible.

Estas pruebas miden el tiempo de respuesta promedio al realizar peticiones a la API, simulando que varios trabajadores realizan consultas de manera simultánea. En este tipo de situaciones se debe validar que la entrada y salida de datos de la base de datos de MySQL maneje de manera eficiente todas las consultas para generar reportes de rendimiento.

También hay que tener en cuenta que la optimización es esencial para asegurar que el sistema funciona incluso en hardware de bajo coste, disminuyendo la descarga de batería y una experiencia de usuario fluida.

Registro de los test

ID	Objetivo	Resultado Esperado	Resultado Obtenido	Fecha de Ejecución	Conclusión
1	Validar la creación de producto con generación automática de QR	Registro en BD exitoso y creación de archivo SVG con UUID	Registro en MySQL confirmado y archivo SVG legible generado	10/03/2026	✓
2	Probar escaneo de QR con cámara nativa de smartphone gama media	Identificación del producto en menos de 2 segundos	Reconocimiento exitoso en 1.5 segundos mediante cámara de 12MP	11/03/2026	✗
3	Verificar que el usuario puede registrarse	Redirección a la página de bienvenida	Redirección a la página de bienvenida	17/03/2026	✓
4	Comprobar que el perfil del usuario se actualiza correctamente	Mensaje de éxito al actualizar	Mensaje de éxito al actualizar	17/03/2026	✓
5	Comprobar no se puede registrar un usuario con un email existente	Mensaje de error: "El email ya está en uso"	Mensaje de error: "El email ya está en uso"	17/03/2026	✓
6	Validar bloqueo de stock negativo en salida de mercancía	Error al intentar retirar más unidades de las existentes	El sistema bloquea la transacción y lanza aviso visual	22/03/2026	✓
7	Verificar reclasificación automática ABC tras simular ventas	El producto cambia de categoría 'B' a 'A' según volumen	Cambio reflejado tras ejecución del script de análisis Pareto	22/03/2026	✓
8	Validar control de acceso por roles (Operador en menú Admin)	El sistema debe denegar el acceso a la sección de usuarios	Acceso bloqueado con error 403 (Acceso Denegado)	23/03/2026	✓
9	Evaluar legibilidad de interfaz en condiciones de luz de almacén	Contraste suficiente para lectura de datos críticos	Cumplimiento visual con guía de estilo "Soft Industrial"	23/03/2026	✓
10	Test de integridad: borrado de producto con movimientos previos	Impedir borrado o realizar borrado lógico para mantener historial	El sistema impide borrado físico; se realiza desactivación lógica	23/03/2026	✓
11	Medir latencia de API en entorno concurrente (10 usuarios)	Tiempo medio de respuesta inferior a 500ms	Latencia promedio de 600ms durante carga constante	23/03/2026	✗

12 Tecnologías utilizadas

Aunque en las fases iniciales de análisis se contempló el uso de Python (Django o Flask) para el backend, el desarrollo final del proyecto se ha llevado a cabo íntegramente en JavaScript, tanto en el lado del servidor como en el cliente. Esta decisión ha permitido unificar el lenguaje de programación en toda la aplicación, simplificando la configuración del entorno y facilitando una integración más fluida entre el frontend y el backend. A continuación se detallan las tecnologías y librerías que conforman el producto final.

12.1 Frontend (cliente)

Tecnología / Librería	Versión	Función en el proyecto
React	18.x	Biblioteca principal para construir la interfaz de usuario mediante componentes reutilizables.
Vite	5.x	Herramienta de construcción y servidor de desarrollo ultrarrápido para el proyecto React.
React Router DOM	6.x	Gestión del enrutamiento en el lado del cliente (SPA), incluyendo rutas protegidas por roles.
Axios	1.x	Cliente HTTP para comunicarse con la API REST del backend. Incluye interceptores para adjuntar el token JWT a las peticiones.
Recharts	2.x	Librería de gráficos para React, utilizada en el dashboard para visualizar la distribución de categorías ABC.
html5-qrcode	2.x	Lectura de códigos QR directamente desde la cámara del dispositivo o desde archivos de imagen.
Lucide React	0.x	Conjunto de iconos vectoriales consistentes con la guía de estilo «Soft Industrial».
Vanilla CSS	-	Hojas de estilo con variables CSS personalizadas para implementar la paleta de colores y diseño responsive.
Validator	13.x	Validación y saneamiento de entradas en el frontend (formularios de registro, login, productos).

12.2 Backend (servidor)

Tecnología / Librería	Versión	Función en el proyecto
Node.js	20.x	Entorno de ejecución de JavaScript en el servidor.
Express	4.x	Framework web minimalista para definir rutas, middlewares y lógica de la API REST.
mysql2	3.x	Conector de MySQL para Node.js. Soporta promesas, prepared statements y pools de conexiones.
jsonwebtoken	9.x	Generación y verificación de JSON Web Tokens para la autenticación sin estado.
bcrypt	5.x	Cifrado de contraseñas con algoritmo bcrypt y 12 rondas de sal.
cors	2.x	Middleware para habilitar Cross-Origin Resource Sharing y permitir peticiones desde el frontend.
helmet	7.x	Protección de cabeceras HTTP frente a vulnerabilidades web comunes.
express-rate-limit	7.x	Limitación de intentos de inicio de sesión para prevenir ataques de fuerza bruta.
cookie-parser	1.x	Parseo de cookies. Se utiliza junto con el almacenamiento del token JWT en el lado del cliente.
dotenv	16.x	Carga de variables de entorno desde un archivo .env.
validator	13.x	Validación y saneamiento de entradas en el backend (reutilizada en las funciones de validación).
UUID (crypto.randomUUID)	-	Generación de identificadores únicos universales para los códigos QR, usando el módulo nativo crypto de Node.js.
qrcode (dependencia de generación externa)	-	La obtención de la imagen QR se realiza mediante redirección a un servicio externo (api.qrserver.com) por simplicidad, si bien el código UUID que la identifica se almacena localmente.

12.3 Base de datos

Tecnología	Función en el proyecto
MySQL (MariaDB 10.4)	Sistema de gestión de base de datos relacional donde se almacenan usuarios, productos, movimientos y alertas. La integridad referencial se garantiza mediante claves foráneas y el motor InnoDB.
phpMyAdmin	Herramienta web para administrar la base de datos durante el desarrollo (creación de tablas, inserción de datos de prueba, copias de seguridad).

12.4 Herramientas de desarrollo y despliegue

Herramienta	Función en el proyecto
Visual Studio Code	IDE principal para la escritura de código, con extensiones para JavaScript, React y control de versiones.
Git y GitHub	Sistema de control de versiones distribuido y repositorio remoto para la gestión del código fuente.
Postman	Cliente para probar manualmente los endpoints de la API durante el desarrollo del backend.
npm	Gestor de paquetes para instalar y administrar las dependencias tanto del frontend como del backend.
ESLint / Prettier	Herramientas de linting y formateo de código para mantener un estilo consistente.

12.5 Nota sobre la elección de JavaScript frente a Python

En las fases de análisis se planteó Python con Django o Flask como opción preferente para el backend, debido a su robustez y a la existencia de librerías como qrcode y Pillow. Durante la implementación, se decidió trasladar toda la lógica a JavaScript (Node.js + Express) por varias razones:

- Permitted unificar el lenguaje en todo el proyecto, eliminando la necesidad de alternar entre Python y JavaScript.
- El ecosistema npm ofrecía alternativas equivalentes a todas las librerías necesarias (qrcode, bcrypt, mysql2, jsonwebtoken).
- La integración con React (frontend) resultó más natural y las herramientas de construcción como Vite ofrecen una excelente experiencia de desarrollo.

13 Manuales

13.1 Manual de usuario

El diseño del sistema de gestión de inventario se basa en la eficiencia operacional y la accesibilidad. El objetivo de este manual es aportar a cualquier usuario una guía detallada sobre cómo interactuar con la plataforma de manera fluida y exprimir al máximo todas las herramientas que aporta.

Requerimientos previos

- Navegador de internet compatible con los estándares de React y la API de captura de vídeo.
- Conexión a Internet estable (4G/5G) que permita la sincronización instantánea de las transacciones con la base de datos de MySQL.
- Hardware de captura de imagen y vídeo que permita la detección de códigos QR sin necesidad de hardware adicional o especializado.
- Autenticación del usuario por medio de un registro autorizado por el administrador para mantener la seguridad y el sistema de control basado en roles.

Perfiles de usuario y niveles de interacción

La plataforma está diseñada de manera que cada usuario posee uno de los 3 roles, los cuales indican la capacidad del usuario para interactuar con el sistema. Los perfiles de usuario son los siguientes:

- Administrador: Tiene control total sobre el sistema y sus funciones incluyen gestión, configuración del sistema ABC de clasificación de productos, supervisión de los reportes financieros y aprobación de los ajustes que sean necesarios en el sistema según la situación.
- Operador: El operador es el usuario que interactúa de manera directa con los productos físicos. Su perfil está orientado a la movilidad y sus tareas prioritarias son el escaneo de códigos QR para registrar cualquier tipo de transferencia y la confirmación de la recepción del stock.

- Auditor: Este perfil está enfocado en la consulta y tienen permiso para visualizar el historial de movimientos y transacciones; y los niveles de stock en tiempo real para generar reportes de rendimiento. Su objetivo en esencia es detectar fallos o discrepancias y verificar la integridad de los procesos aunque no tengan la capacidad de realizar movimientos o modificar los productos disponibles.

13.2 Manual de programador y despliegue

La estructura técnica del sistema de administración de inventarios se basa en una arquitectura moderna y escalable, creada para facilitar su mantenimiento y evolución futura. Este manual permite a los desarrolladores adquirir toda la información requerida para conocer el entorno técnico y el esquema de datos.

Arquitectura Técnica y Stack Tecnológico

El sistema se ha diseñado como un sistema desconectado del frontend y del backend que se comunican a través de una API REST. Esto permite que las capas puedan cambiar y evolucionar independientemente si fuese necesario.

El backend está implementado en Node.js, utilizando el framework Express. Esta elección se debe a que JavaScript es un lenguaje muy extendido, con una gran comunidad y un ecosistema de paquetes (npm) que incluye librerías para la generación de QR, el procesamiento de datos y la conexión con bases de datos. La lógica de persistencia está gestionada por MySQL (MariaDB), un motor de base de datos relacional que permite garantizar la integridad referencial y el rendimiento óptimo para gestionar miles de transacciones simultáneas.

En el frontend se utiliza React como framework para implementar una interfaz de usuario dinámica y reactiva. La arquitectura de componentes de React permite la reutilización de código y que la aplicación sea responsive, de manera que se adapte automáticamente a cualquier tamaño de pantalla.

Entorno de Desarrollo y Configuración Inicial

Para comenzar a trabajar en el proyecto, el desarrollador debe configurar un entorno que incluya las herramientas esenciales de construcción y control de versiones.

Herramienta	Función	Notas de Configuración
VS Code	IDE de Desarrollo	Se recomienda el uso de extensiones para JS y React.
Git / GitHub	Control de Versiones	Gestión de ramas y colaboración en el código fuente.
Docker	Contenedorización	Facilita la creación de entornos idénticos en desarrollo y producción.
npm	Gestores de Paquetes	Instalación de dependencias del frontend y del backend.
Figma	Diseño de Interfaz	Referencia para la implementación de la guía de estilo.

La configuración inicial requiere la instalación de unas librerías específicas. En el backend son importantes comandos como `npm install qrcode`, así como la configuración de los conectores de MySQL mediante `mysql2`. En el frontend se requieren comandos como `npm install` para obtener los módulos de Node.js, incluyendo librerías de escaneo que soportan el uso de cámaras móviles.

14 Conclusión

14.1 Conclusiones

El presente proyecto ha consistido en el diseño, implementación y validación de StockTrace, una aplicación web para la gestión de inventario orientada a pequeñas y medianas empresas, basada en la automatización mediante códigos QR y construida íntegramente con tecnologías de código abierto. Al término de las diecisiete semanas de desarrollo planificadas, se ha obtenido un producto funcional que cubre los cuatro casos de uso definidos en el análisis (gestión de catálogo, registro de movimientos, alertas por stock mínimo y clasificación ABC dinámica) y que respeta el control de acceso jerárquico para los tres perfiles de usuario previstos: administrador, operador y auditor.

En relación con los objetivos específicos planteados al inicio del proyecto, se pueden extraer las siguientes valoraciones:

- Reducción de la tasa de errores por debajo del 1 %. Si bien la métrica exacta requeriría un estudio de campo con usuarios reales durante un periodo prolongado, el diseño del sistema elimina las causas principales de error en la gestión manual de inventarios: la transcripción de datos, el cálculo aritmético del stock y la duplicidad de registros. Toda entrada o salida queda registrada automáticamente, con validaciones tanto en el frontend como en el backend, y las operaciones que violan la lógica de negocio (como intentar retirar más unidades de las existentes) son bloqueadas antes de afectar a la base de datos. La transacción de movimiento, que constituye el núcleo funcional del sistema, se ejecuta bajo un bloqueo de fila (FOR UPDATE) que garantiza la integridad del stock incluso en condiciones de concurrencia.
- Aumento de la eficiencia del proceso logístico. El operador puede completar un movimiento (escaneo del código QR, identificación del producto, selección del tipo y cantidad, y confirmación) en cuestión de segundos, sin necesidad de escribir manualmente referencias ni consultar listados en papel. La interfaz móvil, con el escáner QR integrado, ha sido diseñada precisamente para esta inmediatez, priorizando el acceso a la cámara y la actualización visual instantánea del stock.
- Trazabilidad por unidad mediante códigos QR. Cada producto recibe un identificador único universal (UUID) en el momento de su creación, almacenado en la base de datos y utilizado como semilla para su código QR. Cualquier movimiento posterior queda vinculado al producto y al usuario que lo realizó, lo que permite reconstruir el historial completo de cualquier artículo del almacén. La vista de detalle de producto muestra este historial y la imagen del código QR para su lectura o impresión.

- Eliminación de la dependencia de hardware costoso. La aplicación es plenamente funcional desde un navegador web en un smartphone de gama media. La lectura de códigos QR se realiza mediante la cámara nativa del dispositivo a través de la librería `html5-qrcode`, y también se admite la carga de imágenes o la introducción manual del UUID. En ningún caso se requiere un escáner industrial ni otro periférico especializado.

Desde el punto de vista de los requisitos funcionales, todos los enunciados en el apartado 4.2.1 han sido implementados: el CRUD de catálogo con generación automática de QR, el registro en tiempo real de transacciones, el sistema de alertas automáticas, y el control de acceso basado en permisos con jerarquía de roles. Los requisitos no funcionales también se han satisfecho: la interfaz es responsive y se adapta a dispositivos móviles, la latencia en el reconocimiento de QR y en la actualización del stock es mínima, las contraseñas se almacenan cifradas con `bcrypt` y 12 rondas de sal, y el sistema opera sin dificultad en hardware de bajo coste.

La metodología Kanban ha demostrado ser adecuada para un proyecto de estas características. La organización visual del trabajo mediante un tablero actualizado semanalmente ha permitido identificar con claridad las tareas prioritarias, evitar la acumulación de funcionalidades a medio terminar y reaccionar con flexibilidad ante los ajustes que surgían durante la implementación, tales como la decisión de migrar el backend de Python a Node.js.

Precisamente esta decisión tecnológica merece una mención específica. Aunque los documentos de análisis contemplaban inicialmente Python (Django o Flask), el desarrollo final se ha llevado a cabo íntegramente en JavaScript (Node.js con Express en el servidor y React en el cliente). Este cambio no ha supuesto ninguna merma en las funcionalidades previstas: todos los módulos (autenticación, CRUD, movimientos transaccionales, alertas, clasificación ABC y dashboard) han sido implementados con las mismas garantías de seguridad, rendimiento y mantenibilidad. La unificación del lenguaje en todo el proyecto ha simplificado la configuración del entorno de desarrollo y ha eliminado la fricción que podría producirse al alternar entre dos ecosistemas diferentes.

En el capítulo de pruebas, la batería de tests recogida en el apartado 11 ha permitido verificar aspectos críticos como la generación correcta de códigos QR, el bloqueo de stock negativo, la reclasificación ABC tras simular ventas, el control de acceso por roles y la aplicación de la guía de estilo «Soft Industrial». Los resultados obtenidos, con diez de los once tests cerrados satisfactoriamente, avalan la calidad del producto en su estado actual.

No obstante, el proyecto no está exento de limitaciones. La más evidente es la ausencia de un periodo de prueba con usuarios reales en un entorno de almacén. Aspectos como la usabilidad en condiciones de iluminación variables, la duración de la batería durante el uso continuado del escáner o la percepción subjetiva de la velocidad del sistema solo pueden evaluarse en ese contexto. Asimismo, algunas funcionalidades quedan en estado de prototipo (configuración de preferencias, persistencia de notificaciones, generación de la imagen QR localmente sin dependencia de servicios externos) y no se ha abordado la integración con sistemas externos de facturación o ERP, una demanda previsible en el segmento objetivo.

En el plano personal, la realización de este proyecto ha supuesto la oportunidad de aplicar de forma integrada los conocimientos adquiridos a lo largo del ciclo formativo de Desarrollo de Aplicaciones Web: análisis y especificación de requisitos, diseño de bases de datos relacionales, implementación de una API REST con control de autenticación y autorización, construcción de una interfaz de usuario dinámica con React, y aplicación de metodologías ágiles. El resultado es un producto que, si bien no agota todas las posibilidades de mejora, constituye una base sólida y directamente utilizable por una PYME que desee digitalizar la gestión de su inventario.

14.2 Líneas de trabajo futuras

A partir del estado actual del proyecto, se identifican las siguientes líneas de evolución, ordenadas de menor a mayor complejidad y coste:

- Persistencia de la configuración de usuario. Actualmente, las preferencias de notificaciones y apariencia se almacenan únicamente en el estado local de React y se pierden al cerrar sesión. Una mejora inmediata consistiría en persistirlas en la base de datos (tabla configuracion_usuario) o, para opciones no críticas, en localStorage.

- Generación de la imagen QR en el servidor. La obtención de la imagen del código QR depende hoy de una redirección a un servicio externo (api.qrserver.com). Para garantizar la autonomía del sistema y la privacidad de los datos, se podría integrar la librería qrcode de npm y generar las imágenes en formato PNG o SVG directamente en el backend, sirviéndolas desde un endpoint específico.
- Envío real de correos electrónicos en las alertas. La tabla alerta incluye el campo email_enviado precisamente para este fin, pero la lógica de envío no se ha implementado. Con la integración de nodemailer y un servicio SMTP (o una cuenta transaccional como SendGrid), se podría completar el flujo de notificación previsto en el Caso de Uso 3.
- Reportes de stock anuales y gráficos avanzados. La sección de reportes y estadísticas ofrece actualmente exportación CSV de movimientos y alertas, así como un gráfico ABC en el dashboard. Se podría ampliar con reportes de evolución temporal del stock, productos más y menos rotados, y otros indicadores relevantes para la gestión del almacén.
- Migración a TypeScript. Tanto el frontend como el backend podrían beneficiarse de la adopción de TypeScript. La definición explícita de interfaces para los modelos de datos, las peticiones y las respuestas de la API reduciría los errores en tiempo de desarrollo y mejoraría la documentación automática del código.
- Tests automatizados con frameworks específicos. Las pruebas realizadas durante el proyecto han sido mayoritariamente manuales o semiautomáticas. La incorporación de Jest y React Testing Library (frontend) y Mocha o Vitest (backend) permitiría construir una suite de tests unitarios y de integración que se ejecutara de forma automática antes de cada despliegue.

- Contenedorización con Docker. Aunque se ha documentado su uso, no se ha implementado en el flujo de trabajo real. Disponer de un entorno Dockerizado facilitaría el despliegue en producción, la replicación del entorno de desarrollo en distintos equipos y la escalabilidad horizontal del servicio.
- Funcionamiento offline progresivo (PWA). Una limitación señalada en el DAFO como amenaza es la dependencia de la conexión a Internet. Convertir la aplicación en una Progressive Web App permitiría almacenar en caché los recursos necesarios y encolar movimientos para sincronizarlos cuando se recupere la conexión.
- Integración con sistemas ERP y e-commerce. Una demanda recurrente en el segmento de las PYMEs es la conexión entre el inventario y las ventas. En el futuro, se podría desarrollar una API pública o conectores para plataformas como WooCommerce, Shopify u Odoo, de modo que una venta en línea actualizara automáticamente el stock y viceversa.
- Internacionalización y accesibilidad ampliada. La interfaz está preparada para español e inglés, pero el sistema de traducción es rudimentario. Adoptar una librería como i18next facilitaría la adición de nuevos idiomas y la gestión de textos dinámicos. En paralelo, se podría profundizar en la accesibilidad (WCAG 2.1 AA) con auditorías específicas de contraste, navegación por teclado y compatibilidad con lectores de pantalla.

En conjunto, estas líneas de trabajo no desvirtúan la arquitectura actual, sino que la complementan y la preparan para un uso profesional continuado. La modularidad del código y la separación clara entre el frontend y el backend, defendidas desde las primeras fases del diseño, constituyen la mejor garantía de que StockTrace puede evolucionar sin necesidad de reestructuraciones profundas.

15 Bibliografía

- Packemwms. (2025, 5 noviembre). QR code inventory management: How to unlock efficiency and accuracy. PackemWMS.
<https://packemwms.com/qr-code-inventory-management/>
- Bizowie. (2025, November 26). ABC Inventory Analysis: The complete guide to Strategic Inventory Classification. Bizowie.
<https://bizowie.com/abc-inventory-analysis-the-complete-guide-to-strategic-inventory-classification>
- Bizowie. (2025, November 26). ABC Inventory Analysis: The complete guide to Strategic Inventory Classification. Bizowie.
<https://bizowie.com/abc-inventory-analysis-the-complete-guide-to-strategic-inventory-classification>
- <https://www.ar-racking.com/us/blog/abc-inventory-method-in-the-warehouse-or-%20igin-characteristics-and-advantages/>
- Roger. (2012, January 15). datamodel for a warehouse. Stack Overflow.
<https://stackoverflow.com/questions/8869311/datamodel-for-a-warehouse>
- Nall. (2009, October 6). Database schema for organizing historical stock data. Stack Overflow.
<https://stackoverflow.com/questions/1523576/database-schema-for-organizing-historical-stock-data>
- Jibraiel. (n.d.). Need big help in designing a part of database for a WMS (Warehouse Management System) application : r/Database.
https://www.reddit.com/r/Database/comments/1btc30r/need_big_help_in_designing_a_part_of_database_for/
- Badrah, R. (2026, May 5). WooCommerce & WordPress Glossary | 50+ Terms Explained. WPFactory. <https://wpfactory.com/glossary/>
- Hui, X. (2021, October 29). Implement database schema for organizing historical stock data. Stack Overflow.

<https://stackoverflow.com/questions/69768717/implement-database-schema-for-organizing-historical-stock-data>

- Empresa, S. I. (2025, February 13). Tendencias emergentes para pymes en 2025. Banco Santander Impulsa Empresas.
<https://www.impulsa-empresa.es/tendencias-emergentes-pymes/>
- Fuente, O. (2025, March 15). Tendencias Empresariales para 2025. Thinking for Innovation.
<https://www.iebschool.com/hub/tendencias-empresariales-innovacion-innovacion/>
- Team, G. (2025, January 22). 17 mejores programas de gestión de inventarios [revisados y comparados]. Geekflare Spain.
<https://geekflare.com/es/software/best-inventory-management-software/>
- <https://www.reci.org.mx/index.php/reci/article/download/116/431>
- Shreesh, & Shreesh. (2026, March 3). How can You Use QR Codes for Inventory Management? The QR Code Generator (TQR CG): Create Free QR Codes.
<https://www.the-qrcode-generator.com/es/blog/qr-code-inventory-management>
- Taylor, D., Cabrera, M., & Aranda, A. (2024, March 5). Los 5 mejores software de gestión de inventarios gratis y de código abierto. Capterra.
<https://www.capterra.es/blog/566/software-gestion-inventarios-gratis-y-codigo-abierto>
- Comunicación. (2026, February 11). Odoo precios 2026: licencias, costes y tarifas reales (Community y Enterprise). REIXMOR.
<https://www.reixmor.com/odoo-precios/>
- López, D. V. (2026, March 2). Odoo VS Holded. Nanobytes.
<https://nanobytes.es/blog/comparando-odoo-y-helded-60>
- Todo lo que hay que saber sobre el sistema Verifactu | Sage España. (n.d.). Sistema-verifactu.
<https://www.sage.com/es-es/certificacion-software/sistema-verifactu/>